

Tipos de Datos en C#

Taxonomía profunda, Value Types vs Reference Types, administración del Stack y Heap, y conversión de tipos profesional.

Acerca de este Capítulo

En el Capítulo 1 aprendiste que una variable es un contenedor en memoria. En este capítulo profundizaremos en la clasificación exacta de esos datos: cómo se representan a nivel binario en la RAM, qué diferencia un dato por valor de uno por referencia, cómo realizar conversiones de tipo sin pérdida de información y cómo optimizar el rendimiento del software.

CURSO

Curso de Programación C# NET

ENFOQUE TÉCNICO

Desarrollo de C# y Arquitectura de Memoria

NIVEL EDUCATIVO

Desde Principiante hasta Nivel Práctico

REQUISITOS PREVIOS

Capítulo 1: Variables en C#

1. Objetivo del Capítulo

El objetivo de este capítulo es brindar al estudiante una comprensión rigurosa y práctica de los tipos de datos en C#. Al finalizar este tema, serás capaz de elegir la estructura de datos nativa precisa para cada necesidad de negocio, evitando fallos de rendimiento, fugas de memoria o imprecisiones de cálculo.

Al finalizar este capítulo serás capaz de:

- **Clasificar cualquier tipo de dato en C#** dentro de las categorías de Tipos por Valor (Value Types) o Tipos por Referencia (Reference Types).
- **Explicar el comportamiento de la memoria RAM** diferenciando la pila (Stack) del montón (Heap).
- **Realizar conversiones de tipos (Casting)** de manera implícita, explícita y segura mediante `Parse`, `TryParse` y `Convert`.
- **Evitar desbordamientos numéricos e imprecisiones decimales** mediante el uso correcto de `int`, `long`, `float`, `double` y `decimal`.
- **Trabajar con el tipo nulo (Nullable types)** para representar ausencia de datos en campos de bases de datos o sensores.

2. Introducción al Tema

¿Qué son los Tipos de Datos en C#?

Un **tipo de dato** es una categoría formal que le indica al compilador de C# y al motor de ejecución (.NET CLR) qué clase de información se va a almacenar, cuántos bytes de memoria RAM se deben reservar y qué operaciones matemáticas o lógicas están permitidas sobre esa información.

Analogía de la Vida Real: Los Recipientes de Cocina

Imagina que estás organizando una cocina profesional:

1. Huevera (bool)

Diseñada exclusivamente para 2 estados: o hay huevo o está vacía (`true` / `false`).

2. Taza Medidora (int)

Tiene marcas enteras y fijas para harina o azúcar. No admite líquidos volátiles.

3. Probeta de Precisión (decimal)

Mide con exactitud milimétrica gotas financieras o reactivos químicos sin perder un solo decimal.

¿Por qué existe y qué problema resuelve?

En el nivel más bajo de hardware, la computadora solo comprende secuencias de ceros y unos (`01100001`). Esos 8 bits pueden representar el número `97`, la letra `'a'`, o el estado de activación de un motor. Los **tipos de datos** resuelven la ambigüedad dándole **significado y reglas estrictas** a los patrones de bits almacenados.

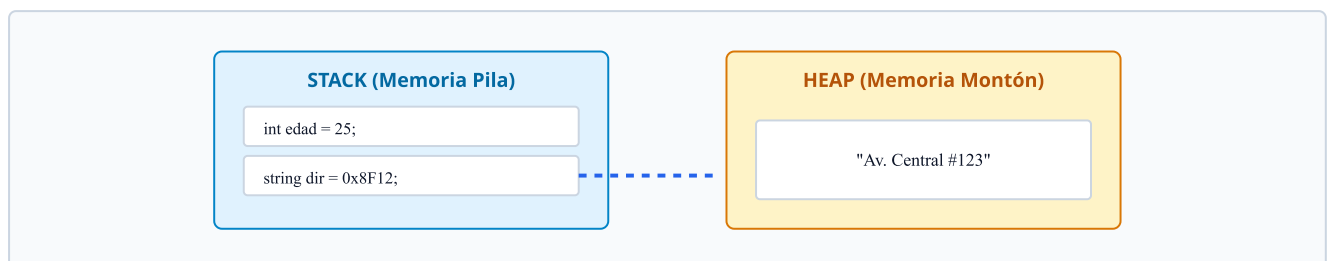
¿Dónde se utiliza en proyectos reales?

La selección precisa del tipo de dato determina la estabilidad de un sistema en producción:

Sistema Real	Requerimiento Técnico	Tipo de Dato Electo	Razón Arquitectural
Bolsa de Valores / Billetera Digital	Transacciones monetarias de millones de dólares.	<code>decimal</code>	Evita errores de redondeo de coma flotante binaria.
Videojuegos (Física / Render 3D)	Cálculo de coordenadas 60 veces por segundo.	<code>float</code> / <code>Vector3</code>	Procesamiento ultrarrápido respaldado por la GPU.
Sensores de Presión IoT	Lectura analógica que puede estar desconectada.	<code>float?</code> (Nullable)	Permite almacenar el valor <code>null</code> cuando falla el sensor.

3. Arquitectura Interna: Pila (Stack) vs Montón (Heap)

C# divide los tipos de datos en dos grandes categorías conceptuales según la forma en que se gestionan dentro de la memoria RAM durante la ejecución:



Tipos por Valor (Value Types)

Se almacenan directamente en la **Stack**. Guardan el dato real. Tienen un tamaño fijo y su liberación de memoria es instantánea cuando la función termina. (Ejemplos: `int`, `double`, `bool`, `char`, `struct`).

Tipos por Referencia (Reference Types)

La variable guarda una *dirección de memoria* en la **Stack** que apunta al objeto real ubicado en la **Heap**. Su recolección la gestiona el Garbage Collector. (Ejemplos: `string`, `object`, `clases`).

4. Taxonomía de Tipos Primitivos Integrados

C# provee palabras clave abreviadas (alias) que mapean directamente a estructuras de la plataforma .NET (System):

Alias C#	Estructura System .NET	Tamaño	Categoría	Rango de Valores
byte	System.Byte	1 Byte (8 bits)	Entero Sin Signo	0 a 255
short	System.Int16	2 Bytes (16 bits)	Entero Con Signo	-32,768 a 32,767
int	System.Int32	4 Bytes (32 bits)	Entero Con Signo	-2,147,483,648 a 2,147,483,647
long	System.Int64	8 Bytes (64 bits)	Entero Con Signo	-9.22×10^{18} a 9.22×10^{18}
float	System.Single	4 Bytes (32 bits)	Coma Flotante IEEE	$\pm 1.5 \times 10^{-45}$ a $\pm 3.4 \times 10^{38}$ (7 dígitos)
double	System.Double	8 Bytes (64 bits)	Coma Flotante IEEE	$\pm 5.0 \times 10^{-324}$ a $\pm 1.7 \times 10^{308}$ (15-17 dígitos)
decimal	System.Decimal	16 Bytes (128 bits)	Decimal Exacto	$\pm 1.0 \times 10^{-28}$ a $\pm 7.92 \times 10^{28}$ (28-29 dígitos)
bool	System.Boolean	1 Byte	Lógico	true / false

5. Tipos Nulables (Nullable Types)

¿Qué es? Por defecto, los tipos por valor (como `int` o `bool`) no pueden ser nulos. Sin embargo, en situaciones reales (como leer datos de un PLC o una base de datos) un valor puede estar ausente.

Sintaxis: Se agrega un signo de interrogación `?` al final del tipo de dato:

```
int? edadOpcional = null; // Permite asignar null o enteros
if (edadOpcional.HasValue)
{
    Console.WriteLine("Edad: " + edadOpcional.Value);
}
```

6. Conversión de Tipos de Datos (Casting)

Convertir datos de un tipo a otro es una tarea cotidiana en software. Existen tres mecanismos principales en C#:

1. Conversión Implícita (Sin Pérdida de Datos)

Ocurre automáticamente cuando se asigna un valor de un tipo más pequeño a uno de mayor capacidad. El compilador no requiere sintaxis especial porque no hay riesgo de truncamiento.

```
int numeroEntero = 1500;
long numeroGrande = numeroEntero; // Conversión implícita automática (int -> long)
double decimalGrande = numeroGrande; // Conversión implícita (long -> double)
```

2. Conversión Explícita (Casting Manual)

Se requiere cuando existe riesgo de pérdida de información (por ejemplo, al pasar de decimal a entero, descartando la parte decimal). Se antepone el tipo deseado entre paréntesis (tipo).

```
double precioExacto = 89.99;
int precioRedondeado = (int)precioExacto; // Resultado: 89 (descarta los decimales)
```

3. Conversión de Texto a Número: Parse y TryParse

Los datos ingresados por el usuario desde la consola o interfaz gráfica llegan como cadenas (string). Debemos transformarlos en valores numéricos para poder operar sobre ellos.

Método int.Parse()

Rápido pero arriesgado. Si el texto contiene letras o está vacío, lanza una excepción que destruye el programa.

```
int x = int.Parse("123");
```

Método int.TryParse() [RECOMENDADO]

Seguro. Retorna true si la conversión tuvo éxito sin generar fallos catastróficos.

```
bool exito = int.TryParse("123", out int x);
```

7. Ejemplos Progresivos de Código

Nivel 1 — Ejemplo Básico: Inspección de Límites de Tipos

Objetivo: Consultar mediante código los valores mínimos y máximos que puede almacenar cada tipo de dato nativo.

```
using System;

class Program
{
    static void Main()
    {
        Console.WriteLine("Rango int: " + int.MinValue + " a " + int.MaxValue);
        Console.WriteLine("Rango byte: " + byte.MinValue + " a " + byte.MaxValue);
        Console.WriteLine("Rango double: " + double.MinValue + " a " + double.MaxValue);
    }
}
```

Explicación Línea por Línea (Nivel 1):

- `int.MinValue` / `int.MaxValue` : Propiedades constantes integradas en la estructura de .NET que retornan los límites numéricos de almacenamiento de la pila.
- `Console.WriteLine(...)` : Convierte los límites a texto y los muestra en la consola.

Nivel 2 — Ejemplo Intermedio: Procesador de Entradas de Usuario Seguras

Objetivo: Capturar la edad de un usuario mediante `Console.ReadLine()` y convertirla de `string` a `int` sin permitir fallos del programa.

```
using System;

class ProcesadorEntrada
{
    static void Main()
    {
        Console.Write("Ingrese la edad del operario: ");
        string entradaTexto = Console.ReadLine();

        // Conversión segura mediante TryParse
        bool esValido = int.TryParse(entradaTexto, out int edad);

        if (esValido)
        {
            Console.WriteLine("Edad registrada exitosamente: " + edad);
        }
        else
        {
            Console.WriteLine("Error: El valor ingresado no es un número entero válido.");
        }
    }
}
```

Nivel 3 — Ejemplo Práctico: Sistema de Control de Calidad Textil

Objetivo: Evaluar métricas de densidad de tela integrando tipos de precisión, enteros y estados nulos.

```
string loteId = "LOT-2026-TX9";
int piezasRevisadas = 1250;
double densidadGramosM2 = 180.45;
decimal costoPorMetro = 12.50m;
float? nivelHumedadSensor = null; // Sensor desconectado por mantenimiento

Console.WriteLine("=== INFORME DE CALIDAD LOTES ===");
Console.WriteLine("Lote: " + loteId + " | Piezas: " + piezasRevisadas);
Console.WriteLine("Costo Total Estimado: $" + (piezasRevisadas * costoPorMetro));

if (nivelHumedadSensor.HasValue) {
    Console.WriteLine("Humedad: " + nivelHumedadSensor.Value + "%");
} else {
    Console.WriteLine("Humedad: [SENSOR NO DISPONIBLE]");
}
```

Nivel 4 — Ejemplo Profesional: Motor de Tarifación de Energía Eléctrica

Contexto: Módulo de cómputo para una empresa distribuidora de energía eléctrica con conversión de unidades y precisión monetaria estricta.

```
using System;

namespace SistemaEnergia
{
    class CalculadoraTarifa
    {
        static void Main()
        {
            // Lecturas analógicas de medidor inteligente
            long idMedidor = 884920491823L;
            double lecturaAnteriorKWh = 14250.80;
            double lecturaActualKWh = 14890.35;

            // Cálculo de consumo
            double consumoTotal = lecturaActualKWh - lecturaAnteriorKWh;

            // Conversiones a finanzas (decimal)
            decimal tarifaPorKWh = 0.1585m;
            decimal subtotal = (decimal)consumoTotal * tarifaPorKWh;
            decimal impuestoIVA = subtotal * 0.16m;
            decimal totalAPagar = subtotal + impuestoIVA;

            Console.WriteLine("=== FACTURACIÓN MEDIDOR #" + idMedidor + " ===");
            Console.WriteLine("Consumo del Periodo: " + consumoTotal + " kWh");
            Console.WriteLine("Subtotal: $" + Math.Round(subtotal, 2));
            Console.WriteLine("TOTAL FACTURA: $" + Math.Round(totalAPagar, 2));
        }
    }
}
```

8. Comparaciones Importantes

Criterio	Tipos por Valor (Value Types)	Tipos por Referencia (Reference Types)
Ubicación Memoria	Almacenados directamente en la Stack .	Dirección en la Stack , objeto en la Heap .
Comportamiento Asignación	Se copia el valor real de la variable.	Se copia la referencia (puntero) al objeto.
Valor por Defecto	0, false, '\0'.	null (ausencia de referencia).
Liberación	Inmediata al salir del ámbito (Scope).	Administrada por el Garbage Collector (GC).

9. Errores Comunes al Trabajar con Tipos de Datos

Error 1: Excepción de Referencia Nula (NullReferenceException)

Intentar acceder a propiedades o métodos de una variable por referencia que apunta a `null`.

```
string cliente = null;
int longitud = cliente.Length; // ¡ERROR FATAL EN EJECUCIÓN!
```

Error 2: Pérdida de Precisión por Coma Flotante Binaria

Sumar montos de dinero usando `double` o `float`. Por ejemplo, `0.1 + 0.2` en `double` produce `0.30000000000000004` debido a la representación binaria IEEE 754. **Solución:** Usar siempre `decimal` para dinero.

Error 3: Desbordamiento Silencioso (Truncamiento en Casting)

Forzar un número grande en un tipo pequeño mediante casting manual: `long valor = 3000000000L; int chico = (int)valor;` La variable `chico` almacenará un número negativo erróneo debido a la pérdida de bits superiores.

10. Buenas Prácticas de Ingeniería de Software

1. Usar TryParse para Entradas

Nunca confíes en que el usuario o la red enviará el formato correcto. Usa siempre `TryParse` en lugar de `Parse`.

2. Usar 'var' con Cuidado

La palabra clave `var` permite inferencia de tipos por el compilador. Úsala únicamente cuando el tipo sea evidente en la asignación.

Recomendación de Tipado Limpio

Prefiere las palabras clave de C# (`int`, `string`, `bool`) sobre los nombres de clase de .NET Framework (`Int32`, `String`, `Boolean`) para mantener consistencia con los estándares de diseño de Microsoft.

11. Conectando Conocimientos

Comprender la naturaleza de los tipos de datos en la RAM amplía significativamente la base construida en el Capítulo 1 y sirve de puente para los siguientes temas:

- **Operadores Aritméticos y Comparación (Capítulo 3):** Verás cómo los tipos numéricos determinan el resultado de divisiones (ej. `5 / 2` da `2` si son enteros, pero `2.5` si son `double`).
- **Programación Orientada a Objetos (Capítulo 5):** Aprenderás a crear tus propios Tipos por Referencia mediante la declaración de `class`.

12. Ejercicios Prácticos (Sin Solución Inmediata)

Ejercicio 1 — Identificación de Tipos Adecuados

Selecciona el tipo de dato nativo óptimo para almacenar: a) El número de planetas del sistema solar, b) La velocidad de la luz en m/s, c) La tasa de interés de una hipoteca, d) La letra de la sección de un asiento en un teatro, e) Si una máquina industrial tiene activo el paro de emergencia.

Ejercicio 2 — Conversor Seguro de Moneda

Escribe un programa que solicite al usuario ingresar una cantidad en Dólares (vía texto) y el tipo de cambio del día. Usa `decimal.TryParse()` para realizar la conversión a Pesos. Si la entrada es inválida, muestra un mensaje descriptivo.

Ejercicio 3 — Manejo de Datos Nulos en Sensores

Declara una variable flotante nutable (`float?`) que simule la lectura de nivel de aceite de una turbina. Escribe un bloque condicional que verifique si la variable contiene un valor; si es nula, asigna un valor por defecto de `0.0f` usando el operador de fusión nula `??` .

Ejercicio 4 — Reto Integrador: Registro de Empleados Enterprise

Diseña un programa que registre: ID de empleado (`long`), Nombre Completo (`string`), Horas Trabajadas (`short`), Tarifa por Hora (`decimal`) e Índice de Eficiencia (`float`). Muestra un informe calculado del salario semanal multiplicando horas por tarifa.

13. Mini Proyecto del Capítulo

Proyecto: Sistema de Telemetría y Control de Calidad Industrial

Contexto: Desarrollarás el motor de validación de datos para una estación de inspección automatizada en una línea de embotellado.

Requisitos Funcionales:

- Capturar la presión de llenado en PSI (`double`) y el volumen en ml (`int`).
- Verificar si el operador activó la inspección manual (`bool`).
- Validar lecturas nulas en caso de falla de comunicación con el PLC.

Pasos Guiados de Implementación:

Paso 1: Definición de Variables de Lote

Declara variables para almacenar el identificador del producto (`string`), la cantidad de botellas procesadas (`int`) y la presión óptima de la línea (`double`).

Paso 2: Lectura Nulable del Sensor PLC

Simula la lectura de un sensor mediante `double? presionSensor = null;`. Aplica el operador `??` para asignar un valor seguro de emergencia (`0.0`) si la lectura resulta nula.

Paso 3: Conversión de Unidades de Volumen

Lee desde texto una cantidad de litros de la tanqueta principal y conviértela usando `float.Parse()` o `TryParse()` a un valor numérico para calcular mililitros totales (multiplicando por 1000).

Paso 4: Reporte de Consola Profesional

Escribe la salida estructurada mostrando el informe de control de calidad sin errores de ejecución.

14. Comprueba tus Conocimientos (Autoevaluación)

Responde las siguientes preguntas antes de verificar las soluciones explicadas en la última página.

1. ¿En qué estructura de memoria RAM (Stack o Heap) se almacena directamente una variable de tipo `int` local?
2. ¿Cuál es la diferencia entre un Tipo por Valor y un Tipo por Referencia en cuanto a lo que guardan en la variable?
3. ¿Por qué una variable de tipo `string` es considerada un Tipo por Referencia a pesar de ser un tipo de dato nativo?
4. ¿Qué ocurre si ejecutas el código `int x = (int)18.95; ?` ¿Cuál es el valor final de `x` ?
5. ¿Qué método de conversión de texto a número previene excepciones cuando la entrada del usuario contiene caracteres no numéricos?
6. ¿Cuál es la sintaxis exacta para declarar una variable entera que acepte valores nulos?
7. ¿Por qué se debe evitar el uso de `float` o `double` al programar transacciones de un banco?
8. ¿Qué realiza internamente la conversión implícita de `short` a `long` ?
9. ¿Qué excepción se produce al intentar invocar un método desde una variable por referencia cuyo valor es `null` ?
10. ¿Cuántos bits de memoria ocupa una variable de tipo `decimal` ?

15. Respuestas y Explicaciones de la Autoevaluación

- 1. Memoria de un Int:** Se almacena en la **Stack (Pila)** debido a que es un tipo de dato por valor.
- 2. Valor vs Referencia:** Un Tipo por Valor almacena los ****datos reales**** en la Stack; un Tipo por Referencia almacena una ****dirección de memoria (puntero)**** que apunta al objeto guardado en la Heap.
- 3. String como Referencia:** Porque un texto puede tener un tamaño dinámico impredecible, lo cual no es óptimo para la Stack (que requiere tamaños fijos en tiempo de compilación).
- 4. Truncamiento en Casting:** El valor final de `x` es `18`. El casting explícito descarta la fracción decimal sin redondear.
- 5. Conversión Segura:** El método `int.TryParse()` o sus variantes para otros tipos numéricos.
- 6. Sintaxis Nulable:** `int? numero = null;` o usando la estructura genérica `Nullable<int> numero = null;`.
- 7. Inprecisión Financiera:** Porque `float` y `double` usan coma flotante binaria IEEE, introduciendo imprecisiones infinitesimales de redondeo inapropiadas para dinero.
- 8. Conversión Implícita:** El compilador expande de manera segura el tamaño de 16 bits a 64 bits sin alterar el signo ni el valor numérico original.
- 9. Excepción por Nulo:** Genera la excepción `System.NullReferenceException`.
- 10. Tamaño de Decimal:** Ocupa exactamente **128 bits (16 Bytes)**.

16. Resumen del Capítulo

Categoría	Tipos Clave	Ubicación Memoria	Uso Principal
Enteros	<code>byte</code> , <code>short</code> , <code>int</code> , <code>long</code>	Stack	Conteo, índices, identificadores.
Flotantes / Decimales	<code>float</code> , <code>double</code> , <code>decimal</code>	Stack	Mediciones físicas (<code>double</code>) / Dinero (<code>decimal</code>).
Texto / Caracteres	<code>char</code> , <code>string</code>	Char: Stack / String: Heap	Símbolos (<code>char</code>) / Cadenas libres (<code>string</code>).
Nulables	T? (ej. <code>int?</code>)	Stack	Lectura de datos opcionales o bases de datos.

Ideas Clave que Debes Recordar:

- C# clasifica los datos en Tipos por Valor (Stack) y Tipos por Referencia (Heap).
- Usa siempre `decimal` para software monetario o contable.
- `TryParse()` es la vía estándar para convertir texto a número de forma segura.
- Los tipos nulables (`?`) permiten gestionar la ausencia de datos en tipos por valor.

17. Mapa de Aprendizaje



¡Excelente progreso! Has completado el Capítulo 2. Estás preparado para abordar el **Capítulo 3: Operadores y Expresiones en C#**.